

AI 에이전트 통신 프로토콜

멀티에이전트 시스템의 대화 체계를 만드는 엔지니어링

AI 에이전트 통신 프로토콜

멀티에이전트 시스템의 대화 체계를 만드는 엔지니어링

ThakiCloud (Hyojung Han)

Contents

1	메시지 포맷의 설계 원칙	1
2	동기 vs 비동기: 언제 무엇을 선택하는가	5
3	상태 공유의 기술	9
4	충돌 해결과 계층적 협업	14

1 메시지 포맷의 설계 원칙

1.1 왜 텍스트가 아니라 구조인가

단일 에이전트가 혼자 동작할 때 메시지 포맷은 큰 문제가 되지 않는다. LLM의 입력 프롬프트를 문자열로 조합하면 그만이다. 그러나 두 개 이상의 에이전트가 서로 대화해야 하는 순간, 문자열 기반의 암묵적 계약은 빠르게 무너진다.

“날씨 조회”라고 보낸 메시지가 결과를 돌려받은 에이전트가 그 결과를 어떻게 해석해야 할지 모르는 상황이 자주 발생한다. 에이전트 A가 “done”이라는 단어를 기대하는데 에이전트 B가 “completed”를 반환하면 파싱이 깨진다. 이 문제를 막는 가장 기본적인 해법은 메시지를 구조화된 데이터로 설계하고 그 구조를 양쪽이 공유하는 것이다.

구조화된 메시지는 세 가지 질문에 답할 수 있어야 한다. 이 메시지의 목적은 무엇인가, 메시지가 담은 실제 데이터는 무엇인가, 그리고 수신자가 결과를 회신할 때 같은 구조를 쓸 것인가. 이 세 질문에 답이 나오면 메시지 설계의 80%는 끝난 셈이다.

1.2 메시지 스키마의 구성 요소

에이전트 간 메시지는 최소한 다음 여덟 가지 필드를 포함해야 한다.

`id`는 메시지의 유일한 식별자다. UUID나 ULID를 사용하고, 응답 메시지가 원본 메시지를 참조할 때 이 ID를 `correlation_id`로 기록한다. 에이전트가 여러 요청을 동시에 처리할 때 이 식별자가 없으면 응답을 어떤 요청에 연결할지 알 수 없다.

`type`은 메시지의 의도를 나타낸다. "request", "response", "event",

"error" 같은 값을 사용한다. type을 "response"로 지정한 메시지는 반드시 correlation_id로 원본 요청을 참조해야 한다.

from과 to는 송신자와 수신자를 식별한다. 에이전트 이름이나 역할 이름을 문자열로 기록한다. broadcast처럼 특정 수신자가 없을 때는 to에 "*"나 "_system"을 사용한다.

payload는 실제 데이터다. 여기에는 에이전트가 업무를 수행하는 데 실제로 필요한 정보가 들어간다. 프롬프트 자체를 payload에 넣을 수도 있지만, 그러면 타입 시스템의 도움을 받지 못하므로 권장하지 않는다.

timestamp는 메시지 생성 시각이다. UTC로 기록하고 ISO 8601 포맷을 사용한다. 메시지가 오래되었는지 판단하는 데 사용된다.

metadata는 부가 정보다. 요청의 우선순위나 응답의 신뢰도 점수, 또는 추적용 trace ID를 넣는다. payload와 분리하면 메시지 라우팅 로그만으로 시스템 전체의 흐름을 파악할 수 있다.

1.3 JSON Schema로 계약 검증 자동화

스키마를 문서로만 남겨두면 시간이 지나면서 구현과 문서가 서서히 어긋난다. 이 갭을 막으려면 스키마를 코드로 관리하고 검증기를 자동으로 돌려야 한다.

Python을 예로 들면, Pydantic 모델을 정의하면 메시지 파싱과 검증이 한번에 처리된다.

```
from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime
import uuid

class AgentMessage(BaseModel):
```

```

id: str = Field(default_factory=lambda: str(uuid.uuid4()))
type: str
from_: str = Field(alias="from")
to: str
payload: dict
timestamp: datetime = Field(default_factory=datetime.utcnow)
metadata: Optional[dict] = None
correlation_id: Optional[str] = None

class Config:
    populate_by_name = True

```

이 모델을 상속받은 메시지만 에이전트 사이를 오갈 수 있게 만들면 잘못된 포맷의 메시지가 시스템에 들어오는 일을 처음부터 막을 수 있다. Type-Script를 쓴다면 Zod나 TypeBox가 같은 역할을 한다.

스키마 변경이 필요할 때는 반드시 버전을 명시한다. `schema_version` 필드를 추가하고, 서로 다른 버전의 메시지가 공존할 때는 수신자가 자체 버전으로 정규화한 뒤 처리하도록 만든다.

1.4 메시지 타입 설계 실전

에이전트 간 통신에서 자주 사용하는 네 가지 메시지 패턴을 정리한다.

먼저 요청-응답이다. 에이전트 A가 에이전트 B에 작업을 요청하고 B가 결과를 반환하는 구조다. `correlation_id`로 요청을 참조하고, 성공하면 `payload`에 결과를, 실패하면 `type: "error"`로 예외를 담아 반환한다.

두 번째는 브로드캐스트다. 에이전트 A가 시스템 전체에 이벤트를 알리는 방식으로, `to: "_system"`으로 전송하면 수신자가 필요에 따라 걸러서 처리한다.

세 번째는 Saga의 단계 메시지다. 긴 업무 프로세스를 여러 에이전트가 이어받아 처리할 때 쓰는데, 각 단계에서는 “이 단계 완료, 다음 단계 시작”이라는 신호만 주고받는다. 전체 프로세스의 상태는 별도 스토어에 저장하고 메시지는 포인터 역할만 맡는다.

마지막은 능동적 알림이다. 사용자 요청 없이 특정 조건이 충족되면 에이전트가 스스로 다른 에이전트에 메시지를 보내는 경우다. 메일함에 새 메시지가 도착했을 때 알림 에이전트가 다른 에이전트에 알림을 전송하는 상황이 대표적이다.

1.5 정리

메시지 포맷을 구조화하면 세 가지 문제가 한꺼번에 풀린다. 수신자가 payload를 해석하는 데 드는 비용이 줄고, 잘못된 포맷이 시스템에 들어오는 걸 검증기가 막으며, 메시지 로그만으로도 에이전트 간 상호작용을 추적할 수 있다. 스키마는 코드로 관리하고 버전을 명시하며 검증은 자동화한다.

2 동기 vs 비동기: 언제 무엇을 선택하는가

2.1 요청-응답 모델의 구조

동기 통신은 에이전트가 메시지를 보내고 응답을 기다리는 가장 직관적인 패턴이다. 에이전트 A가 에이전트 B의 결과가 필요할 때 A는 B를 호출하고 B의 응답이 돌아올 때까지 대기한다.

이 패턴이 맞는 상황은 몇 가지로 나뉜다. 우선 에이전트 A가 에이전트 B의 결과 없이는 다음 단계로 넘어갈 수 없는 경우다. 데이터 검색 에이전트가 웹 검색 결과를 가져와야 분석 에이전트에 전달할 수 있다면 둘 사이의 통신은 동기적이어야 한다.

작업의 완료 시점이 분명해야 하는 경우도 마찬가지다. 사용자의 질문에 최종 답변을 내놓아야 하는 메인 에이전트가 서브 에이전트들의 결과를 모아 회신해야 할 때는 동기 호출이 가장 단순하다.

응답 시간 제한이 있고 그 안에 결과가 반드시 도착해야 하는 경우도 동기가 낫다. 타임아웃을 설정해두면 요청이 실패했다는 걸 바로 알 수 있다.

그러나 요청-응답 모델의 단점도 뚜렷하다. 에이전트 A가 B를 호출하고 대기하는 동안 A는 아무 일도 하지 않고 유향 상태로 남는다. B의 처리가 10초 걸리면 A의 10초가 그대로 낭비되는 셈이다. B가 여러 요청을 동시에 처리해도 A는 각 응답을 순차적으로 기다려야 하니 전체 처리량도 결국 제한된다.

2.2 이벤트 기반 비동기 통신

비동기 통신의 핵심은 에이전트가 메시지를 보낸 후 즉시 다음 작업으로 넘어갈 수 있다는 데 있다. 에이전트 A가 메시지를 발행하면 Broker가 그

메시지를 적절한 수신자에게 전달한다. A는 전달 과정에 관여하지 않는다.

Python에서 간단한 이벤트 버스를 구현하면 다음과 같다.

```
from typing import Callable, Dict, List
from dataclasses import dataclass
import asyncio

@dataclass
class Event:
    topic: str
    payload: dict

class EventBus:
    def __init__(self):
        self._handlers: Dict[str, List[Callable]] = {}

    def subscribe(self, topic: str, handler: Callable):
        self._handlers.setdefault(topic, []).append(handler)

    async def publish(self, event: Event):
        for handler in self._handlers.get(event.topic, []):
            asyncio.create_task(handler(event.payload))
```

이벤트 버스를 쓰면 에이전트 간 결합도가 크게 낮아진다. 에이전트 A는 누가 event를 구독하는지 알 필요가 없다. 새로운 에이전트 B가 기존 코드를 건드리지 않고도 event.topic을 구독하면 자동으로 활성화된다.

비동기가 특히 맞는 상황은 몇 가지로 정리된다. 먼저 에이전트 간 직접 호출이 잦고 그 호출이 체인처럼 이어질 때다. A가 B를 부르고 B가 C를 부르는 구조를 동기로 짜면 전체 대기 시간이 각 단계의 합으로 불어난다. 비동기로 바꾸면 모든 단계를 동시에 시작할 수 있다.

한 에이전트가 여러 에이전트에 동시에 알려야 하는 상황도 마찬가지다.

검색 에이전트가 웹, 문서, 데이터베이스 세 소스에 병렬로 검색을 요청한다면 비동기 브로드캐스트가 유효하다.

작업 시간이 길어서 대기가 시스템 처리량을 떨어뜨릴 때도 비동기가 필요하다. 대용량 파일을 분석하는 에이전트에게 작업을 맡기고 곧바로 다음 요청을 받으려면 비동기 없이는 어렵다.

시스템 확장성이 중요할 때도 그렇다. 동기 호출은 호출 수가 늘어날수록 대기 스레드가 빠르게 바닥난다. 비동기는 적은 스레드로도 많은 동시 연결을 처리한다.

2.3 백프레셔와 결합도 관리

비동기 통신에서 발생하는 가장 흔한 문제는 생산 속도가 소비 속도를 초과할 때다. 에이전트 A가 초당 100개의 메시지를 발행하지만 에이전트 B가 초당 10개만 처리할 수 있다면 메시지가 버스 안에 쌓여 memory를 고갈시킨다.

이 문제를 백프레셔라고 부른다. 해결 방법은 몇 가지가 있다.

우선 흐름 제어 메커니즘을 둔다. 에이전트 B가 처리할 수 있는 메시지 수를 Limited Queue로 관리하다가 큐가 차면 에이전트 A에게 429 Too Many Requests를 반환한다. A는 이때 지수적 백오프로 재시도한다.

메시지 우선순위를 두는 방법도 있다. 모든 메시지가 같은 중요도를 갖는 건 아니다. 사용자의 직접 요청은 백그라운드 태스크보다 먼저 처리해야 한다. Priority Queue를 쓰면 시스템 부하가 높아져도 중요한 메시지는 처리된다.

마지막으로 Dead Letter Queue를 둔다. 재시도 횟수를 넘겨도 처리되지 못한 메시지는 따로 저장해둔다. 나중에 수동으로 분석하거나 재처리할 수 있다.

2.4 동기/비동기 혼합 전략

실제 시스템에서는 두 패턴을 섞어 쓴다. 핵심 설계 원칙은 간단하다. 사용자 요청의 최종 응답 경로에는 동기를 쓰고, 그 경로 밖의 모든 통신에는 비동기를 쓴다.

예를 들어 사용자가 질문하면 메인 에이전트는 검색 에이전트에 동기 호출을 건다. 검색 결과를 받으면 요약 에이전트에도 동기 호출한다. 그러나 검색 에이전트가 실제 웹 검색을 시작하는 순간부터는 내부적으로 비동기 이벤트 버스를 쓴다. 검색 결과를 모을 때도 각 소스의 결과를 기다리는 동안 다른 작업을 병렬로 진행한다.

이렇게 하면 사용자 입장에서 응답 시간은 짧으면서도 시스템 내부 처리량은 높아진다.

2.5 정리

동기와 비동기 중 하나만 선택하는 것은 옳지 않다. 시스템의 응답 시간 요구사항, 에이전트 간 의존성 구조, 처리량 목표를 종합적으로 고려해서 결정해야 한다. 기본 전략은 사용자의 요청 경로에는 동기, 내부 처리의 병렬화가 필요한 부분에는 비동기를 쓰는 것이다.

3 상태 공유의 기술

3.1 왜 에이전트는 상태를 공유해야 하는가

멀티에이전트 시스템에서 각 에이전트가 완전히 독립된 상태만 가지고 동작한다면, 그것은 사실 여러 개의 단일 에이전트를 나열한 것에 불과하다. 진정한 멀티에이전트 협업은 에이전트들이 공유된 정보에 기반해 판단하고 행동할 때 실현된다.

예를 들어 사용자의 세션 정보는 검색 에이전트와 분석 에이전트가 공동으로 참조해야 한다. 검색 에이전트가 사용자가 이전에 질문한 내용을 알면 더 정확한 검색어를 생성할 수 있다. 분석 에이전트도 검색 결과를 참고할 때 검색 에이전트가 어떤 검색을 실행했는지 알면 중복 작업을 덜어낼 수 있다.

그러나 상태 공유에는 근본적인 딜레마가 있다. 공유하는 정보가 많을수록 에이전트 간 결합도가 높아지고 하나의 에이전트가 상태를 잘못 변경하면 시스템 전체에 영향이 퍼진다. 이런 문제를 풀기 위한 검증된 패턴이 몇 가지 있다.

3.2 공유 스토어 패턴

에이전트들이 직접 서로의 상태를 교환하는 대신, 중앙화된 저장소에 읽기 전용 혹은 읽기-쓰기 형태로 접근하는 패턴이다. 에이전트는 상태 변경 이벤트를 스토어에 게시하고, 다른 에이전트는 그 스토어를 구독한다.

Redis를 사용한 구현 예를 살펴본다.

```

import redis, json
from typing import Any

class SharedStore:
    def __init__(self, redis_url: str):
        self.redis = redis.from_url(redis_url)
        self.pubsub = self.redis.pubsub()

    def read(self, key: str) -> dict | None:
        val = self.redis.get(key)
        return json.loads(val) if val else None

    def write(self, key: str, value: Any, version: int) -> bool:
        lua = """
        if redis.call('get', KEYS[1]) then
            local cur = redis.call('get', KEYS[1])
            local cur_ver = json.decode(cur).get('_version', 0)
            if tonumber(cur_ver) >= tonumber(ARGV[2]) then
                return 0
            end
        end
        redis.call('set', KEYS[1], ARGV[1])
        return 1
        """
        ok = self.redis.eval(lua, 1, key, json.dumps(value), version)
        return bool(ok)

```

이 구현에서 핵심이 되는 부분은 버전 관리다. 각 상태 항목에 버전 번호를 붙이고 오래된 버전으로 갱신 요청이 들어오면 그 요청을 거부한다. 이 방식이면 두 에이전트가 동시에 같은 상태를 변경하려 할 때 나중에 도착한 쪽이 실패한다.

공유 스토어 패턴을 쓰면 얻는 이점이 여럿이다. 상태의 출처가 하나로 모이니 추적이 쉬워지고, 스토어를 거쳐 가기 때문에 모든 변경이 감사 로그에 남는다. 에이전트 사이에 스토어라는 간접 계층이 끼어들면서 결합도도 낮아진다. 읽기 전용 접근만 허용한다면 동시성 문제 자체를 피해 갈 수 있다.

3.3 Event Sourcing과 WAL

에이전트의 현재 상태만 저장하면 무엇이 일어났는지 알 수 없다. 사용자가 검색 결과를 요구했는데 검색 에이전트가 이미 종료되어 있었다면, 그 사이 무엇이 있었는지 알 길이 없다.

Event Sourcing은 상태 그 자체가 아니라 상태 변경 event를 저장하는 방식이다. “검색 요청 받음”, “검색 실행 시작”, “검색 결과 15건 획득”, “결과 분석에 전달함” 같은 event를 순서대로 저장한다. 현재 상태는 event 로그를 재생해 재구성한다.

Write-Ahead Log도 이와 비슷한 방식이다. 상태 변경을 저널로 남기고 장애가 발생하면 그 로그를 이용해 상태를 복구한다. 멀티에이전트 환경에서는 이 방식이 특히 쓸모가 있다.

이전 상태로 되돌아갈 수 있다는 점이 우선 눈에 띈다. 분석 에이전트가 잘못된 판단을 내렸다는 사실을 나중에 알게 되면, event 로그를 역순으로 훑어 그 판단 직전 상태부터 다시 시작하면 된다. 감사 측면에서도 event 로그 자체가 처리 이력이 되어 각 에이전트가 어떤 이유로 그런 결정을 내렸는지 역추적할 수 있다. event를 재생하면 다양한 시나리오도 시험해 볼 수 있다. “검색 결과가 달랐다면 분석 결과는 어떻게 바뀌었을까”라는 질문에는 event를 수정해 재실행하는 것으로 답할 수 있다.

event 스키마를 설계할 때는 몇 가지를 반드시 챙겨야 한다. event ID로 각 event를 구분하고 발생 시각으로 순서를 보장하며, 재생하더라도 값이

바뀌지 않는 payload를 설계해야 한다.

3.4 CQRS 적용

Command Query Responsibility Segregation은 읽기 작업과 쓰기 작업을 분리하는 패턴이다. 멀티에이전트 환경에 적용하면 이런 모습이 된다.

쓰기 담당 에이전트는 상태 변경만 처리한다. 분석 결과를 기록하는 에이전트가 검색 결과를 직접 읽지 않는다. 검색 결과를 읽는 것은 오직 검색 에이전트뿐이다. 읽기 담당 에이전트는 필요한 데이터를 요청하고 쓰기 담당 에이전트가 응답하는 형식이다.

이 패턴이 특히 힘을 발휘하는 상황이 몇 가지 있다. 각 에이전트의 전문 분야가 뚜렷할 때가 그렇다. 검색 에이전트는 검색에만 전념하고 분석 에이전트는 분석에만 집중하는 구조라면 CQRS의 분리가 자연스럽게 맞아떨어진다. 읽기 요청이 쓰기 요청보다 훨씬 많은 경우도 마찬가지다. 대다수 에이전트가 상태를 조회하기만 하고 실제 변경은 소수만 한다면, 읽기 전용 뷰를 따로 두어 확장성을 끌어올릴 수 있다. 데이터 정합성이 특히 중요할 때도 이 패턴이 유리한데, 쓰기 경로에만 검증 로직을 넣어도 전체 일관성을 지킬 수 있기 때문이다.

CQRS에서 까다로운 부분은 뷰의 동기화 타이밍이다. 쓰기 에이전트가 상태를 바꾼 뒤에도 읽기 에이전트가 이전 상태를 읽을 수 있는 시간 창이 남아 있기 때문이다. 이를 허용할지 말지는 시스템 요구사항에 따라 판단할 문제다. 허용한다면 최종 일관성 모델로 충분하고 허용하지 않는다면 동기 복제가 필요하다.

3.5 정리

상태 공유에서 가장 중요한 설계 결정은 공유하는 정보의 경계를 어디에 긋느냐다. 에이전트가 참조만 하고 절대 바뀌지 않는 정보라면 읽기 전용

스토어로 공개하는 것만으로 동시성 문제가 사라진다. 변경 가능한 정보는 버전 관리와 event 로그로 감사 가능하고 재현 가능하게 만들어야 한다. 읽기와 쓰기를 분리하면 각 에이전트의 역할이 또렷해지고 시스템의 확장성도 함께 올라간다.

4 충돌 해결과 계층적 협업

4.1 충돌은 왜 발생하는가

멀티에이전트 시스템에서 충돌은 피할 수 없다. 두 에이전트가 동시에 같은 자원을 서로 다르게 바꾸려 하거나, 같은 질문에 상호 모순적인 답을 내놓으면 충돌이 생긴다. 문제는 회피가 아니라 충돌을 어떻게 해결하느냐다.

충돌의 유형은 크게 세 가지로 나뉜다. 동시 쓰기 충돌은 두 에이전트가 같은 자원을 동시에 수정하려는 경우다. 판단 충돌은 같은 입력을 두고 두 에이전트가 서로 다른 결론에 도달하는 경우이고, 우선순위 충돌은 두 에이전트가 각자 자신의 태스크를 더 중요하게 여기는 경우다.

4.2 낙관적 잠금의 실제 적용

비관적 잠금은 자원의 exclusive lock을 먼저 획득한 뒤 그 lock이 풀릴 때까지 다른 에이전트의 접근을 막는다. 구현은 간단하지만 lock을 쥔 에이전트의 작업이 길어지면 나머지 에이전트 전체가 병목을 겪는다.

낙관적 잠금은 반대로 lock을 쓰지 않고 변경 가능한 자원마다 version 번호를 매긴다. 에이전트가 변경을 시도할 때는 현재 버전과 자신이 읽은 시점의 버전이 일치하는지 확인하며, 일치하지 않으면 변경을 거부한다. 거절당한 에이전트는 최신 상태를 다시 읽어 재시도한다.

Python으로 구현한 예는 다음과 같다.

```
class OptimisticStore:
    def update(self, key: str, new_value: dict, read_version: int)
        ↪ -> bool:
```

```

current = self.redis.get(key)
if not current:
    return False
current_data = json.loads(current)
if current_data.get('_version') != read_version:
    return False # 충돌 발생
current_data['_version'] += 1
current_data['data'].update(new_value)
self.redis.set(key, json.dumps(current_data))
return True

```

이 방식은 충돌이 드물고 발생해도 재시도 비용이 낮은 상황에서 효과적이다. 대부분의 멀티에이전트 협업은 충돌 빈도가 낮으므로, 낙관적 잠금이 비관적 잠금보다 더 높은 처리량을 낸다.

충돌이 잦은 상황에서는 승자를 결정하는 방식이 필요하다. 가장 단순한 방법은 우선순위가 높은 에이전트의 변경만 받아들이는 것이고, timestamps로 가장 최신 변경만 유효하게 처리하는 방법도 있다. 더 정교하게는 각 에이전트가 자신의 변경에 confidence 점수를 붙이고 시스템이 가장 높은 점수를 선택하게 하는 방식도 쓴다.

4.3 우선순위 큐와 데드라인 라우팅

에이전트들이 동시에 많은 태스크를 처리하려 할 때는 어떤 태스크를 먼저 실행할지 정해야 한다. 우선순위 큐가 필요한 이유가 여기에 있다.

단순한 우선순위 정렬뿐 아니라 데드라인 기반 라우팅을 함께 쓰면 시스템의 예측 가능성이 올라간다. 각 태스크에 기한을 정해두고, 남은 시간과 필요한 처리 시간을 비교해 기한 내 완료가 불가능한 태스크는 미리 걸러낸다.

구현에서 주의할 점은 우선순위가 태스크의 중요도를 그대로 반영하지는 않는다는 것이다. 긴급하지 않아도 시스템 전체에 중요한 태스크가 있고, 긴급하지만 영향 범위는 작은 태스크도 있다. 시스템 설계자는 이 두 차원을 분명히 구분해야 한다.

4.4 슈퍼바이저 트리 구조

에이전트 수가 늘어나면 전체를 평면적으로 관리하기엔 한계가 있다. 슈퍼바이저 트리는 에이전트를 트리 구조로 묶어 계층적으로 관리하는 패턴이다.

각 슈퍼바이저는 자신 아래의 worker 에이전트를 monitor한다. worker가 실패하면 슈퍼바이저는 세 가지 중 하나를 결정한다. 그 worker를 재시작하거나, 다른 worker에게 작업을 넘기거나, 전체 시스템을 안전한 상태로 되돌린다.

supervisor 전략은 다음과 같이 구분한다.

one_for_one은 worker 하나가 실패하면 그 worker만 재시작한다. 다른 worker에는 영향이 없어서, worker들이 서로 독립적일 때 적합하다.

one_for_all은 worker 하나가 실패하면 모든 worker를 재시작한다. worker들이 서로 상태를 공유해서 하나가 실패하면 다른 worker도 일관되지 않은 상태에 놓였을 가능성이 클 때 적합하다.

rest_for_one은 worker 하나가 실패하면 그 worker와 그보다 하위에 있는 모든 worker를 재시작한다. worker들 사이에 시작 순서 의존 관계가 있을 때 쓴다.

이 패턴의 핵심은 실패 격리 수준을 어디에 두느냐다. 모든 에이전트를 하나의 슈퍼바이저 아래 두면 한 에이전트의 치명적 실패가 전체 시스템을 무너뜨리고, 반대로 슈퍼바이저를 너무 세분화하면 실패 복구 로직이

복잡해진다. 실무에서는 에이전트 역할별로 슈퍼바이저를 두고, 같은 역할 안에서는 one_for_one을, 역할 사이에는 rest_for_one을 적용하는 방식이 효과적이었다.

4.5 멘탈 모델: 협업 계약의 설계

기술적 패턴만으로는 부족하다. 에이전트 간 협업의 멘탈 모델도 세워야 한다. 효과적인 멀티에이전트 시스템에서는 각 에이전트가 다음 세 가지를 분명히 알고 있다.

첫째는 자신의 역할과 책임의 경계다. 어떤 결정은 내가 내리고 어떤 결정은 다른 에이전트에게 넘겨야 하는지 아는 것이다. 둘째는 다른 에이전트의 능력에 대한 기대치로, 검색 에이전트에게 분석을 요청하면 안 된다는 사실을 알고 있어야 한다는 뜻이다. 셋째는 실패했을 때의 행동 규범이다. 다른 에이전트에서 응답이 없거나 오류가 돌아왔을 때 무엇을 해야 하는지를 뜻한다.

이 세 가지를 문서화해두고 시스템 초기화 시점에 각 에이전트가 합의사항을 확인하게 하면 런타임에서 예상치 못한 충돌이 발생할 확률이 줄어든다.

4.6 정리

충돌 해결에서 가장 중요한 것은 충돌 자체를 피하려는 것이 아니라, 충돌이 발생했을 때의 행동 규범을 미리 정해두는 것이다. 낙관적 잠금은 충돌 발생을 감지하고 재시도하는 명확한 contract를 제공하고, 우선순위 큐와 데드라인 라우팅은 시스템 전체의 작업 흐름을 예측 가능하게 만든다. 슈퍼바이저 트리는 실패 격리로 시스템의 회복력을 높인다. 여기에 기술적 패턴 위로 협업 contract를 얹으면, 시스템은 예상치 못한 충돌보다 체계적 설계를 따라 움직이게 된다.